Charter Aircraft Rental API

Backend Guide: how it works, what we used, and why

FastAPI | SQLAlchemy | PostgreSQL | JWT authentication

Version 1.0 | 27 October 2026

Contents

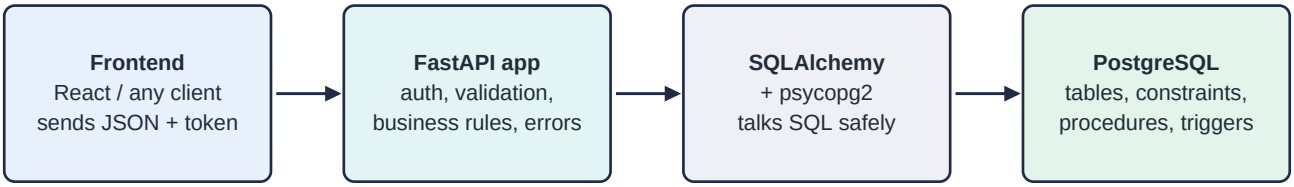
1	The big picture	**9**	Bookings, payments and maintenance
2	Technology stack	**10**	Authentication and roles
3	Project structure	**11**	Security measures
4	The database	**12**	Error handling
5	How a request travels	**13**	API reference
6	Configuration	**14**	Connecting a frontend
7	Input validation	**15**	Running, testing and deploying
8	Master-data endpoints	**16**	Limits and next steps

At a glance

What	Detail
Purpose	A charter-flight booking service: customers search aircraft, book, pay and cancel; staff manage the fleet and keep aircraft serviced.
Language and framework	Python with FastAPI, serving JSON over HTTP.
Database	PostgreSQL: 9 tables (8 original + 1 for logins), 2 functions, 3 procedures, 2 triggers.
Size	About 50 API endpoints and 35 automated tests.
Users	Two roles: **customer** (own data only) and **admin** (staff).
Key idea	The database enforces the hard rules (no double booking, seat limits, no overpayment). The API adds identity, input checking and clear error messages on top.

1. The big picture

The backend is the engine behind a future website or app. The frontend never talks to the database directly. It sends requests to the API, and the API decides whether the caller is allowed to do it, whether the data is valid, and what the database should do.



HTTP (JSON) -> Python -> SQL. Hard data rules live in the database; identity and input checks live in the API.

Who is responsible for what

Layer	Responsibility	Examples
FastAPI app	Identity, permissions, input checks, friendly errors, JSON shape	Is this a valid email? Is this customer allowed to see this booking? Return a clean 409 instead of a crash.
SQLAlchemy	Translates Python objects and queries into safe SQL	Reads and writes rows; all values are sent as parameters, never pasted into SQL.
PostgreSQL	Stores data and enforces the rules that must never be broken	CHECK constraints, foreign keys, the booking procedure, overlap and overpayment triggers.

Why split it this way? Rules inside the database cannot be bypassed, even by a bug in the API or a second program writing to the same tables. The API adds the things a database is bad at: logins, readable errors and checking input before it ever reaches a query.

2. Technology stack

Every tool below is open source. The right-hand column says what it actually does in this project.

Tool	What it is	What it does here
Python 3.14	Programming language	Runs the whole backend.
FastAPI	Web framework	Defines the endpoints, reads requests, builds responses and generates the interactive /docs page automatically.
Uvicorn	Web server	Actually listens on a port and passes requests to FastAPI.
Starlette	Toolkit inside FastAPI	Provides the CORS and trusted-host middleware used for browser safety.
Pydantic v2	Data validation library	Checks every request body and query parameter, cleans the data (trims, lower-cases) and shapes responses.
email-validator	Pydantic add-on	Checks that email addresses are properly formed.
pydantic-settings	Settings loader	Reads configuration from the .env file and environment variables.
SQLAlchemy 2.0	ORM (object-relational mapper)	Maps tables to Python classes so code works with objects; builds the SQL queries.
psycopg2	PostgreSQL driver	The low-level connection between Python and the database.
PostgreSQL 18	Relational database	Stores everything and runs the stored procedures and triggers.
PyJWT	JSON Web Token library	Creates and verifies the signed login tokens.
hashlib.scrypt	Python standard library	Hashes passwords (salted and deliberately slow). No extra package needed.
python-multipart	Form parser	Lets the /docs Authorize button log in with a form post.
pytest + httpx	Testing tools	Run the 35 automated tests, calling the API like a real client would.

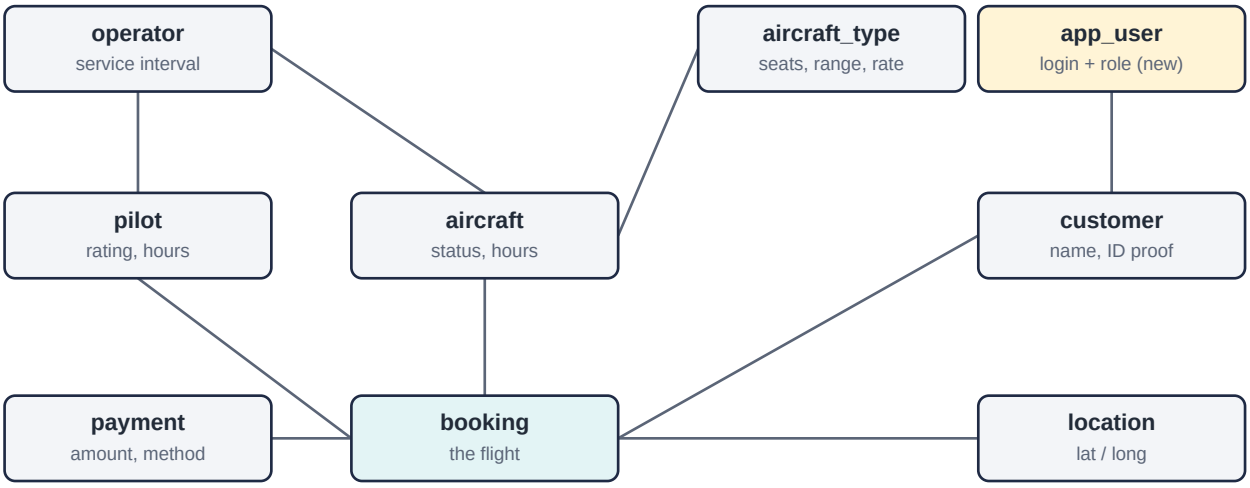
3. Project structure

Each file has one job, so a change in one area rarely touches the others.

File	Purpose
`schema.sql`	Creates the 8 original tables with their constraints.
`routines.sql`	The 2 functions, 3 procedures and 2 triggers that hold the booking rules.
`auth.sql`	Adds the `app_user` table for logins. Kept separate so the original schema is untouched.
`seed.sql`	Demo data (operators, aircraft, locations, pilots) for trying the API.
`app/main.py`	Creates the app, installs middleware and security headers, registers all routes, /health and /meta.
`app/config.py`	All settings in one place, read from .env.
`app/database.py`	The database connection pool and session handling.
`app/models.py`	Python classes that mirror the tables, plus convenience properties (balance due, names).
`app/schemas.py`	Request and response shapes with all the validation rules.
`app/auth.py`	Register, login, token checks and role checks.
`app/security.py`	Password hashing, token creation, rate limiter.
`app/crud.py`	A reusable factory that builds list/get/create/update/delete endpoints for any table.
`app/bookings.py`	Search, quote, bookings, cancel, complete, payments.
`app/fleet.py`	Maintenance, dashboard stats, and the rules guarding fleet and pilot edits.
`app/errors.py`	Turns every kind of failure into one consistent JSON error.
`app/deps.py`	Small shared helpers: DB session, 404 lookup, pagination.
`scripts/`	`setup_db.py` builds the database; `create_admin.py` creates a staff login.
`tests/`	35 tests: validation, the full booking flow, and security.

4. The database

The data model is a classic booking system. A **booking** connects a customer, an aircraft, a pilot and two locations. Payments belong to a booking. One extra table, `app_user`, holds logins and was added for authentication.



Lines show foreign keys. booking ties a customer, an aircraft, a pilot and two locations together; payment belongs to a booking.

The tables

Table	Holds	Notable rules
operator	Charter companies	Unique licence and email; service interval in flying hours (default 100).
aircraft_type	Models (seats, range, speed, hourly rate)	Category is Helicopter, Jet or Turboprop; all numbers must be positive.
location	Airports and helipads	Unique code; latitude and longitude range-checked.
customer	People who book	Unique email; created_on defaults to today.
aircraft	Individual aircraft by registration	Status Available or Maintenance; last service hours can never exceed total hours.
pilot	Pilots belong to an operator	Rated for one category; status Active or On leave.
booking	A flight	Origin and destination must differ; arrival after departure; status Pending, Confirmed, Completed or Cancelled.
payment	Money against a booking	Amount above zero; method Card, UPI or Bank transfer; status Paid or Refunded.
app_user	Logins (new)	Role customer or admin; a customer login links to exactly one customer row.

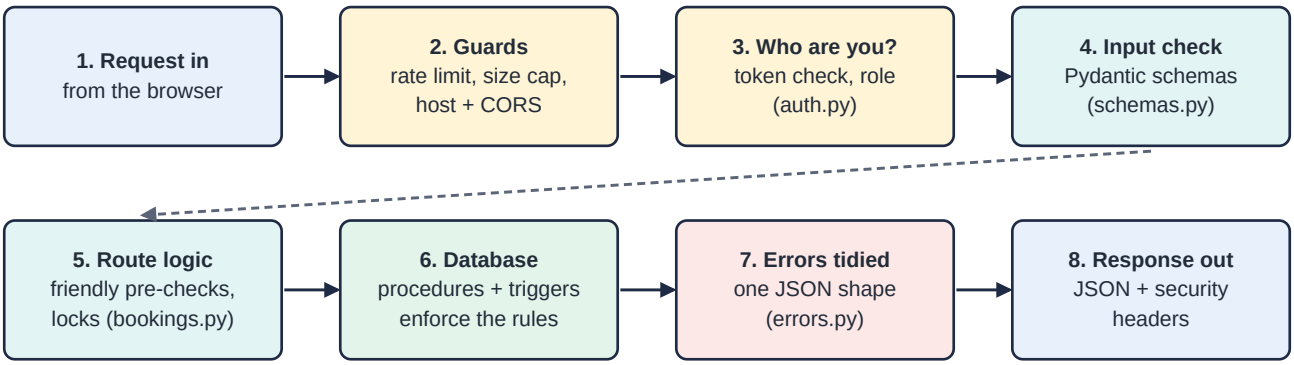
The stored routines (the booking brain)

Routine	Kind	What it does
`fn_distance_nm`	Function	Distance between two locations in nautical miles (haversine formula).
`fn_flight_hours`	Function	Flight time = distance divided by the aircraft's cruise speed.
`book_flight`	Procedure	Checks range, works out flight time, arrival and fare, picks the most experienced free pilot, inserts the booking.
`complete_flight`	Procedure	Marks a Confirmed booking Completed and adds the flown hours to the aircraft and pilot.
`check_overdue_aircraft`	Procedure	Uses a cursor to ground every aircraft past its operator's service interval.
`booking_validate`	Trigger	Before any booking insert: aircraft not in maintenance, seats enough, pilot rating matches, same operator, no overlapping flight.
`payment_confirm`	Trigger	After any payment: rejects overpayment, and confirms the booking once the fare is fully paid.

Fare = flight hours x the aircraft type's hourly rate. The API calls these routines; it does not re-implement them. That keeps one source of truth for the pricing and booking rules.

5. How a request travels

Every call passes through the same sequence. Knowing it makes it easy to see where a failure came from.



- **Guards (step 2)** reject abusive traffic early: more than 300 requests a minute from one address, a body over 64 KB, or a Host header that is not on the allow-list.
- **Auth (step 3)** reads the `Authorization: Bearer ...` header, verifies the token, loads the user from the database and checks the role. Failure gives 401 or 403.
- **Validation (step 4)** compares the body and query string with the schema. Failure gives 422 with a list of field errors.
- **Route logic (step 5)** makes friendly pre-checks (does this aircraft exist? is it in maintenance?), takes locks where two users could collide, then calls the database.
- **Database (step 6)** enforces the rules. A broken rule raises an error carrying a readable message.
- **Error handler (step 7)** converts any failure, including database errors, into the same JSON shape, with no SQL or stack trace shown to the client.
- **Response (step 8)** adds security headers, a request ID and the total-count header on lists.

6. Configuration

All settings come from the `.env` file (never committed to git) or environment variables. An environment variable overrides the file, which is useful to know when debugging.

Setting	Default	Meaning
`DATABASE_URL`	local postgres	Where the database is, including user and password.
`SECRET_KEY`	(required in production)	Signs login tokens. If missing in development, a temporary one is generated and everyone is logged out on restart.
`ENVIRONMENT`	development	Set to production to hide /docs, turn on HSTS and require a SECRET_KEY.
`CORS_ORIGINS`	localhost 3000, 5173, 4200	Which website addresses the browser may call this API from.
`ALLOWED_HOSTS`	localhost, 127.0.0.1	Which Host headers the API accepts (blocks host-header attacks).
`ACCESS_TOKEN_MINUTES`	60	How long a login lasts.
`MIN_LEAD_MINUTES / MAX_ADVANCE_DAYS`	30 / 365	A flight must depart between 30 minutes and 365 days from now.
`RATE_LIMIT_PER_MINUTE`	300	Requests per minute per address, overall.
`AUTH_RATE_LIMIT_PER_MINUTE`	10	Login and register attempts per minute per address.
`MAX_FAILED_LOGINS / LOCKOUT_MINUTES`	5 / 15	Wrong passwords before an account locks, and for how long.

7. Input validation

Validation uses Pydantic schemas in `schemas.py`. There are two kinds on purpose:

- **Input schemas (`*In`)** are strict. They check, clean and reject. Unknown fields are an error, so a client cannot sneak in `"role": "admin"` or `"status": "Confirmed"`.
- **Output schemas (`*Out`)** are plain. They only describe what to return, so an unusual row already in the database can never turn a read into an error.

What gets checked and cleaned

Field	Rule	Example
Email	Valid format, lower-cased, max 100 characters	`Asha@Example.COM` becomes `asha@example.com`
Phone	7 to 15 digits, optional leading +, spaces and dashes allowed	`+91 90000 11111` accepted; `12` rejected
Names and cities	Trimmed, repeated spaces collapsed, not empty, length limited	`Asha Rao` becomes `Asha Rao`
Codes, licences, registrations	Upper-cased; letters, digits and dashes only	`vt-abc` becomes `VT-ABC`
Numbers	Positive and within sensible limits (seats up to 500, speed up to 2000 kts)	Latitude must be -90 to 90.
Money	Maximum 2 decimal places and a digit limit that matches the column	`10.999` rejected
Choices	Only allowed values (category, status, payment method)	`Rocket` rejected as a category
Departure time	30 minutes to 365 days ahead; a timezone offset is converted to server time	Yesterday is rejected with a clear message.
Passwords	10 to 128 characters, at least one letter and one digit, not equal to the email	`short1` rejected

> **One more detail for frontend developers:** numbers in responses (money, hours) are real JSON numbers such as `1250.5`, not strings. Dates and times are plain local time like `2026-11-05T10:00:00`.

8. Master-data endpoints

Operators, aircraft types, locations, customers, aircraft and pilots all behave the same way: list, get one, create, partially update, delete. Instead of writing the same code six times, `crud.py` builds all six sets from one function.

What every list endpoint supports

- `limit` (1 to 500, default 100) and `offset` for paging. The total number of matches comes back in the `X-Total-Count` header, so the frontend can draw page numbers.
- `q` for text search across all text columns. The characters % and _ are treated as plain text, not wildcards.
- Exact-match filters, for example `/aircraft?status=Available&operator_id=2`. They appear in /docs automatically.
- Updates use PATCH: send only the fields to change. The same validation applies and the id cannot be edited.

Business rules guarding edits

Action	Refused when	Why
Set an aircraft back to Available	It is in Maintenance	It must go through POST /aircraft/{id}/service so hours are reset and the cost recorded.
Change an aircraft's operator or type	It has upcoming bookings	The existing bookings were validated against the old values.
Delete an aircraft or pilot	It has upcoming bookings	Cancel those bookings first.
Change a pilot's operator or rating, or set On leave	The pilot has upcoming bookings	Same reason.
Delete anything still referenced	Other rows point to it (foreign key)	Returns 409: "cannot be deleted because booking records still refer to it".

9. Bookings, payments and maintenance

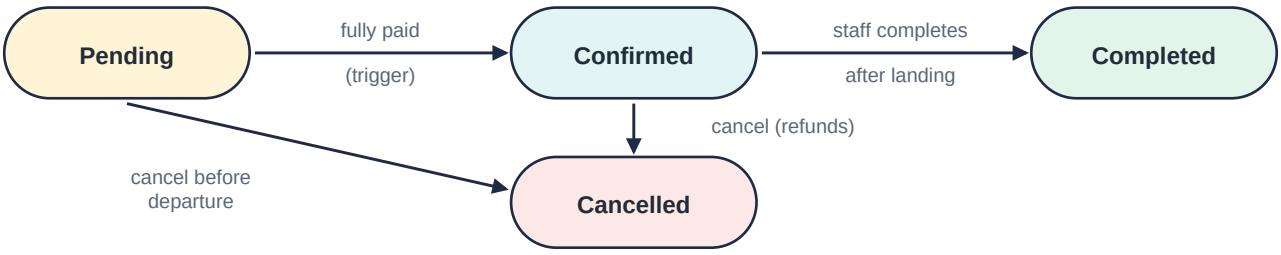
Finding a flight

`GET /search` returns every aircraft that can fly a route at a given time: enough seats, enough range, not in maintenance, no clash with another booking, and with a suitable free pilot, cheapest first. `GET /quote` prices one specific aircraft (distance, flight hours, arrival time, fare) without booking it.

Creating a booking

- The API first confirms the customer, aircraft and both locations exist, the aircraft is Available, and passengers do not exceed seats. This gives quick, readable errors.
- It then takes a short database lock so two people booking at the same moment cannot both get the last slot, and calls the `book_flight` procedure.
- The procedure checks range, computes arrival and fare, picks the most experienced free pilot of the same operator and rating, and inserts the booking. The trigger re-validates everything.
- Customers book for themselves (leave out `customer_id`). Staff must say who the booking is for.

The booking life cycle



- **Pay:** part payments are allowed. Overpayment is refused. When payments add up to the fare, the trigger flips the booking to Confirmed.
- **Cancel:** allowed while Pending or Confirmed and before departure. Every Paid payment becomes Refunded.
- **Complete (staff only):** only a Confirmed booking whose flight has landed. It adds the flown hours to the aircraft and pilot, then automatically grounds any aircraft past its service interval.

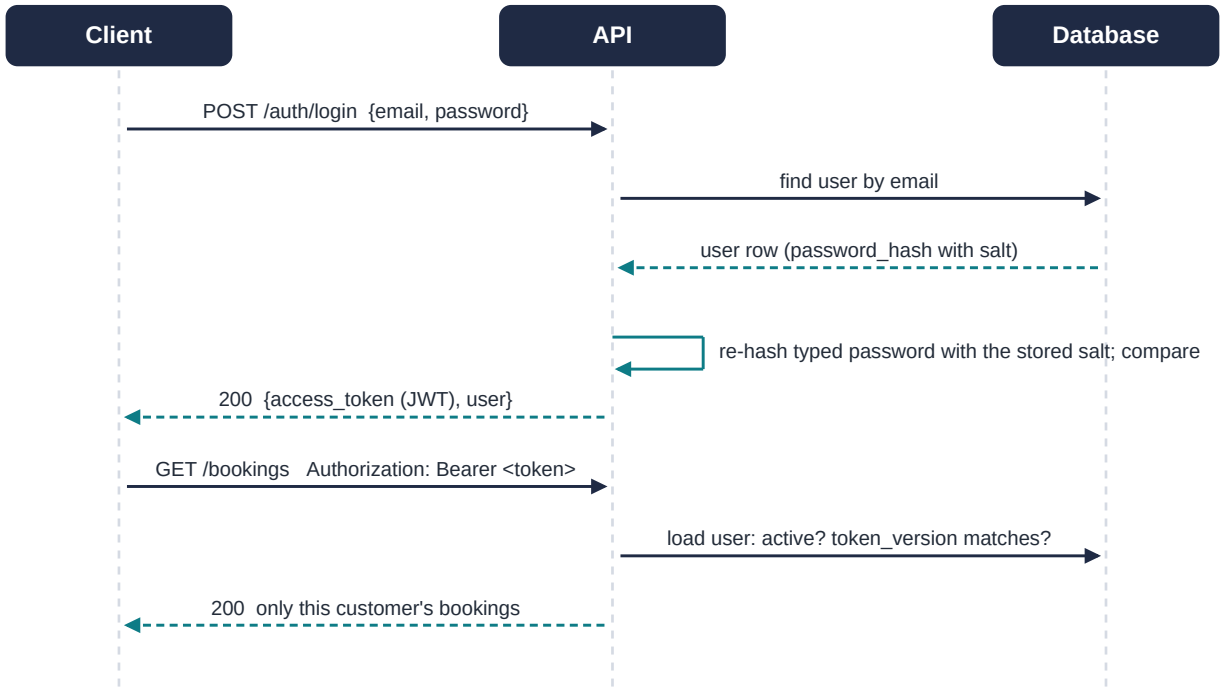
Maintenance

`POST /aircraft/{id}/service` records a service: hours-since-service resets to zero, the cost is added, the service date is saved and the aircraft returns to Available. `POST /maintenance/check` grounds all overdue aircraft on demand, and `GET /stats` gives a dashboard (bookings by status, revenue collected, outstanding balance, aircraft needing service soon).

> **Safe under pressure:** operations that change money or hours (pay, cancel, complete) lock the booking row first. Two simultaneous requests therefore cannot both succeed, so hours are never added twice and payments never slip past the fare.

10. Authentication and roles

Authentication answers *who are you*; authorization answers *what may you do*. The login sequence below is all a frontend has to follow.



The two roles

	Logged out	Customer	Staff (admin)
Locations, /meta, /health	Yes	Yes	Yes
Aircraft types, search, quote	No	Yes	Yes
Book, pay, cancel, list bookings	No	Own bookings only	Everything
Customers, operators, aircraft, pilots	No	No	Yes
Complete flights, maintenance, stats, any create/edit/delete	No	No	Yes

Anyone can register, but only as a customer. Staff accounts exist only if you create them with `scripts/create_admin.py`. If a customer asks for someone else's booking, the API answers 404 (not 403), so they cannot even tell it exists.

Passwords

When an account is created, `hash_password` in `security.py` generates a random 16-byte **salt**, runs the password and salt through **scrypt** (a deliberately slow, memory-hungry hash) and stores one string in `app_user.password_hash`:

```
scrypt$16384$8$1$<salt, base64>$<hash, base64>
```

The salt is stored beside the hash on purpose: it makes identical passwords produce different hashes and defeats precomputed tables. On login the stored salt is read back, the typed password is hashed the same way and the two hashes are compared in constant time. The real password is never stored or logged, so even the database owner cannot read it.

Login tokens (JWT)

A successful login returns a signed token with three dot-separated parts: header, payload, signature. The payload holds the user id, a token version and issue/expiry times. It is **signed, not encrypted**, so it carries no secrets, and nobody can alter it without the server's **SECRET_KEY** invalidating the signature. The client sends it as `Authorization: Bearer <token>` on every call. The server keeps no session list; it verifies the signature, then loads the user so that a disabled account or changed password takes effect immediately.

Brute-force protection

- 5 wrong passwords in a row lock that account for 15 minutes, even for the correct password.
- Login and register are limited to 10 attempts a minute per address.
- Unknown email, wrong password and disabled account all return the identical message, and take about the same time, so attackers learn nothing.
- Changing a password raises the token version, which signs out every other device.

11. Security measures

Each row pairs a common attack with the defence in this project.

Threat	Defence
Stolen password database	Passwords stored only as salted scrypt hashes.
Guessing passwords	Account lockout, per-address rate limit on login, password strength rules.
Forged or tampered token	Signature check with a secret key; the algorithm is fixed on the server, so the "alg: none" trick fails.
Old token after password change	Token version is checked on every request.
Customer reading another's data	Every booking and payment route checks ownership; others get 404.
Customer promoting themselves	Unknown fields are rejected, so `role` and `status` cannot be sent.
SQL injection	All queries use bound parameters; search text is escaped. Tests prove a classic injection string is stored as plain text.
Abusive traffic	Global rate limit, 64 KB body cap, pagination cap of 500.
Malicious websites calling the API	CORS lists only your frontend origins and allows only needed methods and headers; tokens are in a header, not cookies, so there is no CSRF risk.
Host-header attacks	Trusted-host allow-list.
Information leaks	Errors never show SQL, stack traces or database names; logs hold path and status only, never passwords, tokens or query strings.

Threat	Defence
Browser-side risks	Headers: nosniff, no framing, no-store caching, strict Content-Security-Policy; HSTS in production.
Exposed documentation	In production /docs, /redoc and the OpenAPI file are switched off.
Leaked secrets in git	.env and virtual environments are git-ignored; setup scripts never print passwords.

12. Error handling

Every error, from any source, has the same shape so the frontend needs just one handler:

```
{ "detail": "Passengers (9) exceed the aircraft seats (6)" }

{ "detail": "Invalid input - email: value is not a valid email address",
  "errors": [ { "field": "email", "message": "value is not a valid email address" } ] }
```

Status	Meaning	Typical cause
400	Bad request	Request origin not allowed by CORS or host not trusted.
401	Not logged in	Missing, expired or invalid token; wrong email or password.
403	Not allowed	A customer calling a staff endpoint; wrong current password.
404	Not found	No such id, or a booking that belongs to someone else.
409	Conflict or rule broken	Duplicate email, overlapping booking, aircraft in maintenance, overpayment, wrong booking state, deleting something still in use.
413	Body too large	Request over 64 KB.
422	Invalid input	Failed validation; the `errors` list names each field.
429	Too many requests	Rate limit or account lockout; see the Retry-After header.
500	Our fault	Unexpected bug. Generic message only; details go to the server log.
503	Database unavailable	The database cannot be reached.

Database errors are translated rather than exposed. A duplicate key becomes *"A customer with email 'x' already exists"*; a rule raised inside a procedure (such as a double booking) arrives as its own readable sentence.

13. API reference

Roles: **Public** = no login, **User** = any logged-in account, **Staff** = admin only, **Owner** = the customer who owns it, or staff. Interactive documentation with a Try-it-out button is at `/docs` when running in development.

Endpoint	Does	Who
`POST /auth/register`	Create a customer account and log in	Public
`POST /auth/login`	Email and password (JSON) for a token	Public
`POST /auth/token`	Same, as a form (used by the /docs Authorize button)	Public
`GET /auth/me`	Current user, role and customer profile	User
`PATCH /auth/me`	Customer edits own name, phone, ID proof	User
`POST /auth/change-password`	Change password; signs out other devices	User

Endpoint	Does	Who
`GET /locations`, `/locations/{id}`	Airports and helipads	Public
`GET /aircraft-types`, `/{id}`	Aircraft catalogue	User
`GET /operators, /aircraft, /pilots, /customers`	Lists and single records, with filters and search	Staff
`POST, PATCH, DELETE` on all six master tables	Create, edit, delete master data	Staff
`GET /search`	Aircraft that can fly a route at a time	User
`GET /quote`	Distance, time and fare for one aircraft	User
`GET /bookings`	List (customers see only their own); filters for customer, aircraft, pilot, status, dates	User
`GET /bookings/{id}`	One booking with names, paid and balance due	Owner
`POST /bookings`	Create a booking (starts Pending)	User
`POST /bookings/{id}/cancel`	Cancel and refund payments	Owner
`POST /bookings/{id}/complete`	Mark flown, add hours, ground overdue aircraft	Staff
`GET /bookings/{id}/payments`	Payments on a booking	Owner
`POST /bookings/{id}/payments`	Record a payment	Owner
`POST /aircraft/{id}/service`	Record a service and return to Available	Staff
`POST /maintenance/check`	Ground all overdue aircraft	Staff
`GET /stats`	Dashboard numbers	Staff
`GET /meta`	Dropdown lists and rules for forms	Public
`GET /health`	Is the API, database and routines ready?	Public

14. Connecting a frontend

The five things a frontend must do

- **Log in** with `POST /auth/login` and keep the `access_token` in memory (or sessionStorage).
- **Attach the token** to every request: `Authorization: Bearer <token>`.
- **Read the role** from `GET /auth/me` to decide whether to show customer screens or staff screens.
- **Handle errors** with one function: show `detail`, mark fields from `errors` on 422, and send the user to the login page on 401.
- **Use** `GET /meta` to fill dropdowns and display rules (categories, payment methods, minimum lead time) instead of hard-coding them.

Example (JavaScript)

```
const API = "http://localhost:8000";

async function login(email, password) {
  const r = await fetch(`${API}/auth/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });
  if (!r.ok) throw new Error((await r.json()).detail);
  return (await r.json()).access_token;
}

async function myBookings(token) {
  const r = await fetch(`${API}/bookings?limit=20`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  const total = r.headers.get("X-Total-Count");   // for page numbers
  return { items: await r.json(), total: Number(total) };
}
```

A typical customer journey

Screen	Calls
Sign up / log in	`POST /auth/register` or `POST /auth/login`
Pick route and date	`GET /locations`, then `GET /search?origin_id&dest_id&departure_time&passengers`
Confirm	`GET /quote` (optional), then `POST /bookings` returns the booking with the fare
Pay	`POST /bookings/{id}/payments` until `balance_due` is 0; status turns Confirmed
My trips	`GET /bookings` and `POST /bookings/{id}/cancel`

Browser calls blocked? The frontend's address must be listed in `CORS_ORIGINS` in the backend's .env, including the port (for example http://localhost:5173). Restart the API after changing it.

15. Running, testing and deploying

First-time setup

```
python -m venv venv
.\venv\Scripts\python.exe -m pip install -r requirements.txt
copy .env.example .env                        # then set DATABASE_URL and SECRET_KEY, and save
.\venv\Scripts\python.exe scripts\setup_db.py --seed
.\venv\Scripts\python.exe scripts\create_admin.py you@example.com
.\venv\Scripts\python.exe -m uvicorn app.main:app --reload
# then open http://localhost:8000/docs and click Authorize
```

setup_db.py creates the database if needed, loads the schema, routines and login table only where missing, and with `--seed` adds demo data. It is safe to rerun. `--reset` wipes everything and is destructive.

Tests

File	Needs a database?	Covers
`test_validation.py`	No	Input rules, normalisation, error shape, CORS, security headers, oversized and malformed requests.
`test_flow.py`	Yes	The whole journey: search, quote, book, pay in parts, confirm, complete, ground for service, cancel and refund, duplicates, guards.
`test_security.py`	Yes	Logged-out access, role limits, ownership, token forgery and expiry, lockout, rate limits, injection attempts.

Run the database tests against a throwaway database (for example `charter_test`), because they create rows. Setting `DATABASE_URL` in the terminal overrides the .env file for that session.

Before going live

- Serve the API behind HTTPS (a reverse proxy such as nginx or Caddy). Without it, passwords and tokens travel unencrypted.
- Set `ENVIRONMENT=production`, your own `SECRET_KEY`, the real `ALLOWED_HOSTS` and `CORS_ORIGINS`.
- Run uvicorn with `--proxy-headers` so rate limits see real client addresses.
- Pin dependency versions in requirements.txt and back up the database regularly.

16. Limits and next steps

The backend is complete for a working booking system. These are the known edges, so nothing comes as a surprise later.

Area	Current state	Suggested next step
Payments	A payment is only recorded; there is no real gateway.	Integrate a provider (Razorpay, Stripe) and confirm payments by webhook.
Password reset and email check	Not built.	Add email verification and a reset link flow.
Session length	A 60-minute token; users log in again after that.	Add refresh tokens for longer sessions.
Rate limiter	Counts live in one process's memory.	Move to Redis or the reverse proxy if running several workers.
Unpaid bookings	A Pending booking holds its slot indefinitely.	Auto-cancel unpaid bookings after a time limit (needs a scheduled job).
Booking lock	One global lock serialises booking creation.	Lock per aircraft and pilot if traffic grows.
Time zones	Times are local server time without a zone.	Store UTC and convert in the frontend if you serve several regions.
Audit trail	No history of who changed what.	Add an audit table fed by the API or by triggers.